

OSSR

Open Stacks Sponsor Relay

Technical explanation · Transaction sponsorship and fee policy

Prepared October 5, 2026 · Repository snapshot bcd186b and inspected working-tree sources

OSSR lets a user send sBTC on Stacks without first holding STX. The user signs the intended action; a relay operator supplies the sponsor authorization and pays the Stacks network fee in STX. A Clarity adapter pays that operator a quoted sBTC fee in the same transaction that transfers sBTC to the recipient.

The three amounts to keep separate

Amount	Who pays / receives	Unit
Transfer amount	User → recipient	sBTC sats
Sponsor reimbursement	User → actual transaction sponsor	sBTC sats
Stacks network fee	Sponsor → Stacks fee mechanism	microSTX

One sBTC sat is 10^{-8} sBTC; one STX is 1,000,000 microSTX. OSSR provides fee abstraction above the existing Stacks sponsorship mechanism. The on-chain network fee remains STX-denominated.

Default sponsor fee: minimum 2 sats

For every positive transfer, the default logarithmic curve starts at **2 sats**. Live network costs and operator allowances can increase the quote. The 2-sat amount is a minimum for the default policy, not a promise that every transaction costs exactly 2 sats.

Implementation scope

The inspected implementation is a single-relay, testnet proof of concept for one allowlisted action: **sbtc-sponsored-transfer-v1.sponsored-transfer**. The relay is TypeScript/Node.js; the operator desktop uses Rust/Tauri. Local JSON files store quotes and sponsor nonce reservations. Rust relay and PostgreSQL designs in older architecture documents are future targets, not the current execution path.

The broader goal is independent operators and explicit token adapters. Multi-operator routing, mainnet operation, arbitrary application calls, additional tokens and v0.2 batching must not be inferred from the demonstrated v0.1 transfer flow.

1. End-to-end transaction flow

Wallet / user	Relay operator	Stacks
GET /v1/info		
POST /v1/quotes ----->	fee + signed quote	
<-----		
sign origin transaction		
POST /v1/sponsorships -->	validate + nonce lock	
	sponsor-sign	
	simulate signed bytes -> node	
	broadcast ----->	
poll transaction status		
	adapter executes atomically	
	recipient + sponsor paid	

1 • Obtain metadata and a quote

The wallet reads relay identity, testnet configuration, sponsor principal, quote key, pinned adapter/token and pricing limits from /v1/info. It requests a quote for an origin, recipient, amount, optional memo and maxSponsorFeeSats. The relay calculates the fee, checks the user ceiling, creates a random quote ID and persists the signed offer. The default quote lifetime is 10 Stacks blocks.

2 • Build and sign the user authorization

The client builds a sponsored contract call with six arguments: amount, recipient, sponsor-fee, quote-id, expiry-height and memo. It includes the configured sBTC fungible-token post-condition for exactly amount + sponsor-fee in Deny mode. The origin signs first. The relay never needs the wallet's private key to sponsor that action.

3 • Validate, sponsor and simulate

The relay decodes the submitted bytes and checks origin authorization, network, allowlisted call, quote-bound arguments and exact asset post-condition. It checks operator readiness and estimates the STX fee from the fee rate and transaction byte size. A request exceeding the network-fee policy is rejected. Quote reservation binds the quote to a hash of the submitted transaction.

Under serialized sponsor nonce ownership, the operator adds its nonce, STX fee and sponsor signature.

The current code simulates the fully signed transaction before broadcasting. Authenticated Stacks Core simulation must identify those exact transaction bytes, use a sufficiently fresh tip, report no VM/post-condition abort and return (ok true). Failed, stale or malformed simulation stops broadcast.

4 • Broadcast and observe settlement

After successful simulation, the operator persists the nonce reservation and broadcasts. The API returns the transaction ID and STX fee; the client polls /v1/sponsorships/{txid}. Submission is not confirmation. Successful on-chain adapter execution settles both sBTC transfers, while an abort settles neither token transfer.

2. Atomic settlement and quote security

What the Clarity adapter enforces

The adapter rejects mainnet, missing sponsors, expired quotes, zero amounts/fees and amounts above its limits. The current source caps a transfer at **10,000,000 sats** and sponsor reimbursement at **1,000 sats**. The origin cannot equal the recipient or sponsor. Expiry is inclusive: stacks-block-height must be less than or equal to expiry-height.

```
sponsor = unwrap(tx-sponsor?)
assert transfer and fee limits; assert unexpired
transfer sponsor-fee from tx-sender to sponsor
transfer amount from tx-sender to recipient
print ossr-sponsored-transfer event
return (ok true)
```

The sponsor recipient comes from **tx-sponsor?**, rather than a caller-supplied payout address. Both token calls must succeed; failure rolls back the contract effects. There is no escrow or user balance ledger in this adapter and no separate reimbursement transfer in the configured atomic path. CLI setup disables the legacy reimbursement worker when the adapter is configured, preventing a second payout.

Signed offer and exact intent binding

Quotes use SIP-018 signed structured data over Clarity values, with domain name ossr-quote, version 1 and testnet chain ID 2147483648. Signed fields include quote and relay IDs, origin and sponsor, network, action, token identity/unit/precision, adapter and function, argument hash, exact sponsor fee, maximum STX fee, issue/expiry heights, policy version and key ID.

The argument hash commits to a Clarity tuple containing amount, recipient, sponsor-fee, quote-id, expiry-height and memo. Changing any of these invalidates the binding. Large monetary values use bigint arithmetic and decimal strings in JSON to avoid unsafe JavaScript integer conversion.

Responsibility boundaries

The contract checks settlement constraints and emits a quote-linked event; it does **not** verify the SIP-018 signature or independently calculate exchange rates. Relay-side quote state and transaction validation enforce the commercial offer. The origin signature fixes the action, and the post-condition constrains sBTC outflow.

The intended client protocol verifies quotes against trusted relay keys before signing. The documented prototype gap is that its CLI/browser helpers do not independently perform that verification. HTTPS/relay trust currently supplies quote authenticity to the demo client; this remains a production requirement.

Abort economics

Atomicity protects token settlement, but it does not refund an STX fee already charged for an included aborted transaction. The sponsor can lose STX without receiving sBTC. Simulation reduces this exposure but cannot reserve future chain state or guarantee execution at mining time.

3. Fee policy and the 2-sat minimum

For a positive transfer A in sats, default scale $S = 100$ and growth $G = 2$. The relay computes an integer logarithm by finding the smallest exponent whose power of two reaches the input; it does not use a floating-point logarithm.

```
U = ceil(A / 100)
L = 2 × ceil(log2(1 + U))
F = max(1, L, C)
```

A = transfer amount; L = curve fee
 C = operator cost floor; F = quoted fee in sats

Why the minimum is 2 sats: $A \geq 1$ implies $U \geq 1$, so $\text{ceil}(\log_2(1 + U)) \geq 1$ and $L \geq 2$. Therefore the default formula is equivalently $F = \max(2, L, C)$ for valid positive transfers. For 1–100 sats the curve fee is 2 sats; higher cost floors still take precedence.

Transfer	Default curve fee	Fee / transfer
1 sat	2 sats	200%
100 sats	2 sats	2%
1,000 sats	8 sats	0.8%
10,000 sats	14 sats	0.14%
100,000 sats	20 sats	0.02%
1,000,000 sats	28 sats	0.0028%
10,000,000 sats	34 sats	0.00034%

These are curve examples, assuming $C \leq L$. The final quote can be higher. Absolute fees rise in steps while the fee as a percentage falls. Small transfers can therefore carry a large percentage fee.

Live cost floor

```
E = fee_rate_microSTX_per_byte × estimated_bytes
N = ceil(E × STX_USD × 100,000,000
        / (1,000,000 × BTC_USD))
C = N + infrastructure + risk_reserve + minimum_profit
```

The default size estimate is 600 bytes; infrastructure and risk allowances default to 0 sats and minimum profit to 1 sat. The live fee rate must be positive, so network cost rounds up to at least 1 sat and the default dynamic floor is at least 2 sats. The code supports a direct STX/BTC price feed (Coinbase by default) or STX/BTC USD inputs, normalized to eight decimal places before integer conversion.

Live pricing is cached for 60 seconds. Failed or invalid dependencies produce `PRICING_UNAVAILABLE` instead of a quote. The quote's cost estimate differs from the later network fee estimated using actual transaction size; the configured floor and reserves protect estimates, not guaranteed realized profit.

4. Overrides, user limits and operations

Preserving a strict 2-sat minimum

The current generic implementation uses $\max(1, \text{requestedFee}, \text{costFloor})$, and the Clarity contract requires a positive fee rather than enforcing 2 sats. Thus **2 sats is the default-policy minimum, not a universal contract invariant**. A fixed override or smaller growth value with dynamic pricing disabled can permit a 1-sat quote.

For a policy that must always retain a 2-sat minimum, use the default growth of 2 and/or configure `SBTC_BREAK_EVEN_FEE_SATS=2` when dynamic pricing is disabled. Keep the fixed override at least 2 sats. With valid dynamic pricing enabled, the positive converted network cost plus minimum profit already creates a floor of at least 2 sats.

Fixed fees and operator ownership

`SBTC_SPONSOR_FEE_SATS` replaces the curve with a fixed requested fee, but the cost floor still applies: $F = \max(1, \text{fixedFee}, C)$. Disabling dynamic pricing substitutes `SBTC_BREAK_EVEN_FEE_SATS`, whose code default is 1, for C . The operator receives the entire quoted sBTC reimbursement. The atomic v0.1 flow has no protocol treasury fee or fee split.

An older reimbursement specification describes a 10-sat minimum, a 5% markup and a 2-sat reserve. Those figures describe the earlier policy, not the current default logarithmic fee engine. The recorded acceptance run used 10-sat reimbursements and should not be interpreted as establishing the default minimum.

User protection and worked example

The quote request includes `maxSponsorFeeSats`. If the calculated fee exceeds that ceiling, the relay returns `SPONSOR_FEE_TOO_HIGH` (HTTP 422). It must obtain a newly approved price rather than silently increase the limit. Metadata provides a preview; the signed quote provides the exact fee used to build the transaction.

Example: sending 1,000 sats yields a curve fee of 8 sats. With a 3-sat cost floor, the quoted fee is 8 sats and total user outflow is 1,008 sats. With a 12-sat floor, it becomes 12 sats and total outflow 1,012 sats. If the user's maximum is 10 sats, the latter quote is rejected. The relay pays a separate STX network fee in either successful case.

Persistence and duplicate requests

Quotes and nonce reservations are durably stored in owner-only local JSON files for one relay process. Identical completed submissions return the stored result; changed bytes for a reserved quote are rejected. In-progress requests return a conflict. Ambiguous broadcast results retain reservations for operator reconciliation, avoiding blind signing of another transaction.

Endpoint	Purpose
GET /v1/info	Identity, quote key and pricing policy
POST /v1/quotes	Issue signed transfer quote
POST /v1/sponsorships	Validate, sponsor, simulate and broadcast
GET /v1/sponsorships/{txid}	Observe chain status
GET /health/live · /health/ready	Liveness and operator readiness
GET /v1/metrics	JSON operational metrics

5. Evidence, limits and source references

Recorded testnet evidence

The repository's September 19, 2026 status records nine consecutive acceptance transfers by a user holding zero microSTX. Each sent 100 sats to the recipient and reimbursed the sponsor 10 sats. Across those nine transactions, the sponsor paid 125,664 microSTX and received 90 sats. The deployed adapter had ten canonical successful calls in total, including earlier activity.

The controlled rejection run records four rejected cases, zero broadcasts and no fee/token movement. These are documented historical acceptance results, not a new live verification performed for this report.

Production work still required

The prototype is testnet-only and unaudited. It relies on ordinary process-held keys, one relay/sponsor, local JSON persistence and external RPC/pricing availability. It does not provide cross-host locking, automated ambiguous-broadcast recovery, configurable confirmation depth or reorg-aware accounting. Status metrics count observed chain success and are process-local rather than durable settlement accounting.

A production deployment needs independently reviewed contracts, trusted-key verification in clients, an isolated signer, transactional durable storage, redundancy, recovery/reconciliation, confirmation-depth policy and a secured HTTP perimeter. Multi-token integrations require explicit adapters and reviewed precision, post-condition and pricing rules. Proposed batching is a separate v0.2 design, not a current v0.1 capability.

Sources used for this report

Implementation takes precedence where older design documents differ. Paths below are relative to the OSSR repository. The report was prepared from locally inspected sources; no live chain or market prices were used.

apps/operator/src/api.ts

Quote signing, intent validation, pricing formulas, routes and simulation response checks.

apps/operator/src/operator.ts

Sponsor authorization, nonce serialization, simulation callback and broadcast.

apps/operator/src/cli.ts

Runtime configuration and disabling the legacy worker in atomic mode.

apps/operator/src/quote-store.ts; nonce-store.ts

Quote idempotency and durable local reservation behavior.

contracts/sbtc-sponsored-transfer-v1.clar

Pinned sBTC token, atomic transfers, expiry, limits and event fields.

docs/specs/default-fee-policy.md

Logarithmic pricing policy, cost floor, previews and user fee ceiling.

docs/specs/quote-format.md

SIP-018 intent and quote protocol design.

docs/PREGRANT-LIMITATIONS.md

Known prototype risks and client quote-verification gap.

README.md; docs/TESTNET-DEPLOYMENT.md

Recorded testnet acceptance outcomes and deployment evidence.

docs/ARCHITECTURE.md; docs/specs/reimbursement.md

Historical target architecture and earlier reimbursement policy; distinguished from current code.

Repository: github.com/ossr-protocol/ossr